দশমিক থেকে বাইনারি, এবং পরিমাপের একক

পাঠ 2 (চলমান) — 1IOT, SST-ICT-01 — ধারাবাহিক ভাগ, bit/byte/word

16/09/2026

1 দশমিক থেকে বাইনারি: ধারাবাহিক ভাগের পদ্ধতি

আগের পাঠে আমরা বাইনারি সংখ্যাকে দশমিকে রূপান্তর করেছি। এবার দেখি **উল্টো পথ**।

সংজ্ঞা

একটি দশমিক সংখ্যাকে বাইনারিতে রূপান্তর করতে **বারবার 2 দিয়ে ভাগ করা হয়**, প্রতিবার **ভাগশেষ** (**resto**, 0 বা 1) লিখে রাখা হয়, যতক্ষণ না **ভাগফল** (**quoziente**) **0** হয়। বাইনারি সংখ্যাটি পড়া হয় **শেষ ভাগশেষ থেকে প্রথমটি পর্যন্ত** (**নিচ থেকে ওপরে**)।

উদাহরণ

39-কে বাইনারিতে রূপান্তর করা:

ভাগ	ভাগফল	ভাগশেষ	
$39 \div 2$	19	1	(শেষ বিট)
$19 \div 2$	9	1	
$9 \div 2$	4	1	
$4 \div 2$	2	0	
$2 \div 2$	1	0	
$1 \div 2$	0	1	(প্রথম বিট)

ভাগশেষগুলো **নিচ থেকে ওপরে** পড়ে: 100111_2।

যাচাই (আগের পাঠে যেমন দেখেছি): $100111_2 = 32 + 0 + 0 + 4 + 2 + 1 = 39$। ঠিক আছে!

টীকা

সবসময় ফলাফল যাচাই করো: একটি সংখ্যা রূপান্তর করার পর, সেটিকে উল্টো দিকে **আবার রূপান্তর** করে দেখো শুরুর সংখ্যাটি ফিরে আসে কিনা। **হিসাবের ভুল** ধরার এটিই সবচেয়ে সহজ উপায় – ক্লাসেও এমন হয়েছে, একটি বড় সংখ্যার (500-600-এর কাছাকাছি) দ্বিতীয় উদাহরণে হিসাবের সময় একটি অঙ্ক হারিয়ে গিয়েছিল, আর মোটকে শুরুর মানের সাথে মিলিয়ে দেখার সময়ই ভুলটি ধরা পড়ে।

উদাহরণ

অনুশীলনের জন্য তৃতীয় একটি উদাহরণ: **192**-কে বাইনারিতে রূপান্তর করা।

ভাগ	ভাগফল	ভাগশেষ	
$192 \div 2$	96	0	
$96 \div 2$	48	0	
$48 \div 2$	24	0	
$24 \div 2$	12	0	
$12 \div 2$	6	0	
$6 \div 2$	3	0	
$3 \div 2$	1	1	
$1 \div 2$	0	1	(প্রথম বিট)

ভাগশেষগুলো **নিচ থেকে ওপরে** পড়ে: 11000000_2 – শুধু **সবচেয়ে গুরুত্বপূর্ণ** দুটি **বিট** 1, বাকি সব 0।

2 Bit, byte, word: জিনিসগুলোর নাম

সংজ্ঞা

বিট (BInary digiT, "বাইনারি অঙ্ক") হলো কম্পিউটার যে **সবচেয়ে ছোট তথ্য** নিয়ে কাজ করতে পারে: 0 বা 1। এর চেয়ে ছোট কিছু নেই – বিট হলো ডিজিটাল তথ্যের "**পরমাণু**"।

উদাহরণ

সব বিট 1 থাকা একটি বাইট (11111111_2) দশমিক সংখ্যা **255**-এর সমান – 8 বিটে প্রকাশযোগ্য **সর্বোচ্চ মান**, যা $2^8 = 256$টি সম্ভাব্য সংমিশ্রণের (0 থেকে 255) সাথে মেলে।

সংজ্ঞা

8 বিটের একটি দলকে বলে **বাইট** (**byte**)। **2 বাইটের** (16 বিট) একটি দলকে বলে **word**। আরও বড় দলের জন্য সাধারণত সরাসরি **বিটের সংখ্যা** বলা হয় (যেমন "32 বিটের একটি রেজিস্টার")।

টীকা

বাস্তবে একটি একক বিট নিয়ে খুব কমই কাজ করা হয়: প্রায় সবসময় **বাইট স্তরে** কাজ হয়। তাই আমাদের জন্য **পরিমাপের একক** হলো **বাইট**, যদিও ধারণা হিসেবে বিটই সবচেয়ে ছোট একক।

2.1 কতগুলো "বাক্স" দরকার? ভেরিয়েবলের আকার

ভেরিয়েবলের (**variabili**, মেমোরির সেই "**ড্রয়ার**" যেখানে আমাদের ডেটা থাকে) আকার আলাদা হতে পারে, সবসময় **বাইটের গুণিতকে**: 1 বাইট (8 বিট, 255 পর্যন্ত), 2 বাইট (16 বিট, 65.535 পর্যন্ত), 4 বাইট (32 বিট, প্রায় 4,3 বিলিয়ন পর্যন্ত), আধুনিক কম্পিউটারে 8 বাইট (64 বিট) পর্যন্ত।

সতর্কতা

দরকারের চেয়ে **বড়** **ভেরিয়েবল** বেছে নেওয়া **ভুল** **নয়**, কিন্তু এতে **মেমোরি** **নষ্ট** **হয়**। কোনো ডেটার মান যদি সর্বোচ্চ 15 হয় (যেমন একটি ক্লাসের ছাত্রসংখ্যা), তাহলে 32 বিটের ভেরিয়েবল লাগে না: 8 বিটই যথেষ্টের বেশি। ভেরিয়েবলের আকার **অপ্টিমাইজ** **করা** একটি **ভালো** **অভ্যাস**, ছোটখাটো ব্যাপার নয়।

3 ভেরিয়েবল: মেমোরির "বাক্স"

এবার **প্রোগ্রামিং** নিয়ে কথা বলি। প্রোগ্রাম করতে আমাদের বিট, কিন্তু প্রধানত **বাইট** নিয়ে কাজ করতে হয়। প্রোগ্রাম হলো **নির্দেশের** **একটি** **ক্রমবদ্ধ** **তালিকা** – কিন্তু কম্পিউটার আসলে কী করে? বাস্তবে কম্পিউটার **একটিমাত্র** **মৌলিক** **কাজ** জানে: **তথ্য** **নিয়ে** **কাজ** **করা** (বিট)। বাকি সব কিছু (খেলা, হিসাব, ছবি দেখানো) এই একটি মৌলিক ক্ষমতাকে ক্রমশ জটিল স্তরে জুড়ে তৈরি।

হিসাব করতে হলে **সংখ্যা** লাগে – আর আমরা দেখেছি কম্পিউটার সেগুলোকে ভেতরে **বাইনারি** রূপে রাখে। কিন্তু এই সংখ্যাগুলো কম্পিউটারের ভেতরে কোথায় "রাখা" হয়? প্রোগ্রাম করা মানে, অন্যান্য কাজের মধ্যে, কম্পিউটারের ভেতরে কিছু ভার্চুয়াল "**বাক্স**" তৈরি ও চিহ্নিত করা, যেখানে আমরা সংখ্যা রাখি।

টীকা

ড্রয়ারের তুলনা: তোমার ঘরের আলমারির কথা ভাবো, যাতে কয়েকটি **ড্রয়ার** আছে – একটি মোজার, একটি গেঞ্জির, একটি অন্তর্বাসের, একটি প্যান্টের। প্রতিটি ড্রয়ারে কিছু জিনিস থাকে এবং তার একটি নাম থাকে (যেমন "মোজার ড্রয়ার", যাতে হয়তো 12টি মোজা আছে)। এই আধারগুলো হলো সেই আধারের মতো যেখানে কম্পিউটার **ডেটা**, তথ্য, হিসাবে ব্যবহৃত সংখ্যা রাখে।

সংজ্ঞা

এই "ড্রয়ারগুলোর" **কারিগরি** **নাম** হলো **ভেরিয়েবল** (**variabile**): ভেরিয়েবল হলো কম্পিউটারের **মেমোরিতে** **একটি** **ড্রয়ার**, যেখানে আমরা তথ্য রাখতে পারি – আর এই তথ্য **সবসময়** **শুধু** **সংখ্যা**। ভার্চুয়াল ড্রয়ারে আমরা সত্যিকারের একটি অন্তর্বাস রাখতে পারি না: শুধু আমাদের কাছে থাকা অন্তর্বাসের **সংখ্যা** রাখতে পারি (যেমন 3, 7, 42)।

উদাহরণ

বাক্সগুলোর **আকার** **আলাদা** হতে পারে। **1** **বাইটের** বাক্সে থাকে **0** **থেকে** **255** পর্যন্ত সংখ্যা ($2^8 = 256$টি সংমিশ্রণ)। এটি যেমন প্যান্টের জোড়ার সংখ্যার জন্য যথেষ্ট (কারও 255-এর বেশি থাকে না), কিন্তু যেমন এক প্লেট রিসোত্তোর চালের দানার সংখ্যার জন্য **খুব** **ছোট** (255 দানা হবে হাস্যকর পরিমাণ)।

তাহলে দরকার **একটি** **বড়** **বাক্স**: **2** **বাইটের** **(16** **বিট)**। সাবধান: বাক্সের আকার দ্বিগুণ করলে প্রকাশযোগ্য **সর্বোচ্চ** **সংখ্যা** **দ্বিগুণ** **হয়** **না** – 16 বিটে $2^{16} =$ **65.536**টি **সম্ভাব্য** **সংমিশ্রণ** (0 থেকে 65.535), যা শুধু দ্বিগুণের চেয়ে **অনেক** **বড়** **লাফ**।

আরও বড় সংখ্যার জন্য **4** **বাইটের** **(32** **বিট)** বাক্স: 2^{32} দেয় প্রায় 4 বিলিয়ন 300 মিলিয়ন (4.294.967.296) সম্ভাব্য সংমিশ্রণ। বাক্স **8** **বাইট** **(64** **বিট)** পর্যন্তও হতে পারে: প্রায় $1{,}8 \times 10^{19}$টি সংমিশ্রণ, সাধারণ ব্যবহারের জন্য বিশাল একটি সংখ্যা।

টীকা

তোমার বাড়ির কম্পিউটার আজকাল সাধারণত **64** **বিটের** **ভেরিয়েবল** তৈরি করে: ভেতরের মৌলিক কাজে 1 বা 2 বাইটের বাক্স নিয়ে খুব কমই কাজ করে। ইতিহাসে **প্রচলিত** **আকার** **সময়ের** **সাথে** **বেড়েছে**: 15-20 বছর আগে কম্পিউটার কাজ করত **32** **বিটে**, 40 বছর আগে 16 বিটে, 50-60 বছর আগে 8 বিটে।

4 পরিমাপের একক: kilo, mega, giga, tera...

সংজ্ঞা

বিপুল পরিমাণ বাইট মাপার **প্রচলিত** **গুণিতকগুলো** **1000-এর** **ঘাত** (10^3) ধরে এগোয়: **kilo** (10^3), **mega** (10^6), **giga** (10^9), **tera** (10^{12}), **peta** (10^{15}), **exa** (10^{18}), **zetta** (10^{21}), **yotta** (10^{24}) এবং নতুন নামকরণে **ronna** (10^{27}) ও আরও। সবচেয়ে চরম উপসর্গগুলোর সঠিক নাম যাচাই করতে সবচেয়ে সুবিধাজনক উৎস হলো **পরিমাপের** **উপসর্গ** নিয়ে Wikipedia-র পাতা।

উপসর্গ	প্রতীক	পূর্ণ সংখ্যা	10-এর ঘাত
kilo	K	1.000	10^3
mega	M	1.000.000	10^6
giga	G	1.000.000.000	10^9
tera	T	1.000.000.000.000	10^{12}
peta	P	1.000.000.000.000.000	10^{15}
exa	E	1.000.000.000.000.000.000	10^{18}
zetta	Z	1.000.000.000.000.000.000.000	10^{21}
yotta	Y	1.000.000.000.000.000.000.000.000	10^{24}

উদাহরণ

4 **terabyte** (TB) ডিস্কে থাকে প্রায় 4×10^{12} বাইট, অর্থাৎ প্রায় 32 হাজার বিলিয়ন বিট ($4 \times 8 = 32 \times 10^{12}$)। কম্পিউটার চলার জন্য প্রতিটি বিটের মান সঠিক হতে হবে: এমন একটি ব্যবস্থা, যার জন্য প্রায় নিখুঁত **নির্মাণ-সূক্ষ্মতা** দরকার।

টীকা

উপসর্গের একই যুক্তি "উল্টোভাবেও" ব্যবহার হয়, **খুব ছোট পরিমাণের** জন্য: **milli** (10^{-3}, এক হাজার ভাগের এক), **micro** (10^{-6}), **nano** (10^{-9}), **pico** (10^{-12})। যেমন, একটি **ন্যানোমিটার** পরমাণুর আকারের মাত্রার কাছাকাছি।

উপসর্গ	প্রতীক	পূর্ণ সংখ্যা	10-এর ঘাত
milli	m	0,001	10^{-3}
micro	μ	0,000001	10^{-6}
nano	n	0,000000001	10^{-9}
pico	p	0,000000000001	10^{-12}

5 এই পাঠ থেকে যা মনে রাখতে হবে

- **দশমিককে বাইনারিতে** রূপান্তর করতে: **2 দিয়ে ধারাবাহিক ভাগ**, **ভাগশেষগুলো** নিচ থেকে ওপরে পড়ে।
- **বিট** (BInary digiT) হলো সবচেয়ে ছোট ডিজিটাল তথ্য (0/1); **বাইট** হলো 8 বিটের দল; **word** হলো 2 বাইটের দল।
- **ভেরিয়েবল** হলো মেমোরির একটি "ড্রয়ার" যেখানে আমরা সংখ্যা রাখি (**আর কিছু কখনো নয়**): ভেরিয়েবলের **আকার বাইটের গুণিতক** (1, 2, 4, 8 বাইট), আর **উপযুক্ত সবচেয়ে ছোটটি** বেছে নিলে **মেমোরি নষ্ট** হয় না।
- **গুণিতকগুলো** (kilo, mega, giga, tera…) এগোয় **1000-এর ঘাতে** (প্রতি লাফে 10^3)।
- রূপান্তরের হিসাব সবসময় উল্টো দিকে **আবার রূপান্তর** করে **যাচাই** করো।

উদাহরণ

পরের পাঠের জন্য করণীয়: **ধারাবাহিক ভাগের** পদ্ধতিতে 100 সংখ্যাটিকে বাইনারিতে রূপান্তর করো, তারপর দশমিকে ফিরিয়ে ফলাফল যাচাই করো।

6 শব্দকোষ: ক্লাসে ব্যবহৃত ইতালীয় শব্দ

- **divisione**: ভাগ
- **quoziente / resto**: ভাগফল / ভাগশেষ
- **dal basso verso l'alto**: নিচ থেকে ওপরে

- **verifica / verificare**: যাচাই / যাচাই করা (**verifica** মানে পরীক্ষাও)
- **variabile**: ভেরিয়েবল
- **cassetto / scatola**: ড্রয়ার / বাক্স
- **memoria**: মেমোরি
- **dati**: ডেটা
- **prefisso / simbolo**: উপসর্গ / প্রতীক
- **miliardi**: বিলিয়ন (1.000.000.000)